Evaluation and Conversion of Expression

Expression Evaluation:

Expression evaluation is one of the fundamental topics in Data Structure and Algorithms. An expression is a combination of operators and operands representing specific values. For example, in a + b, **a** and **b** are operands, and + is an operator.

Expression evaluation is the process of computing the final numerical or logical value of a mathematical or programming expression. It involves substituting variable values, applying operations (such as arithmetic or logical operations), and following strict rules of operator precedence and associativity.

Generally, we cannot directly use the BODMAS rule concept to evaluate expressions in a computer system. Therefore, operator precedence and associativity rules are used to evaluate expressions correctly.

Operator Precedence:

When multiple operators occur in an expression, the concept used to decide the order in which the operators will be applied and evaluated first is called operator precedence.

Key Concept

Higher and Lower Precedence Rule:

When an operator has higher precedence than other operators, it will be applied and evaluated first. Operators with lower precedence are applied and evaluated later, after the higher precedence operators have been completely evaluated.

Parentheses ( ):

If parentheses occur in an expression, they override the normal precedence rules and force the evaluation of the expression inside them first. The operators inside the parentheses are evaluated before the others, regardless of their precedence.

If two or more parentheses exist in an expression, they are evaluated from left to right. In the case of nested parentheses, the innermost parentheses are evaluated first, while the outermost ones are evaluated last.

Operator Associativity:

When multiple operators in an expression belong to the same precedence level, it becomes unclear which one should be evaluated first. The concept that determines the direction of evaluation in such cases is called operator associativity.

There are two types of operator associativity:

- Left to Right Associativity
- Right to Left Associativity

Arithmetic Expression

An expression that contains only arithmetic operands and arithmetic operators is called an arithmetic expression.

Types of Arithmetic Expression

There are three main types of arithmetic expressions, namely: Infix, Prefix, and Postfix.

1. Infix Expression:
When an operator is placed between the operands, such a mathematical expression is called an infix expression.
It is expressed as:
Infix Exp = (opnd1) op (opnd2)
Examples:
A + B
X + C * D
2. Prefix Expression:
When an operator is placed before the operands, such a mathematical expression is called a prefix expression. It is also called Polish Notation.
It is expressed as:
Prefix Exp = op (opnd1) (opnd2)
Examples:
+AB
*+CD
3. Postfix Expression:
When an operator is placed **after** the operands, such a mathematical expression is called a **postfix expression**.
It is also called Reverse Polish Notation.
It is expressed as:
Postfix Exp = (opnd1) (opnd2) op
Examples:
AB+
DF/+

<u>Order of Precedence and Associativity Table (PEDMAS Rule)</u>

Operator	**Precedence/ Priority**	**Associativity**
( ), { }, [ ]	1	Left to Right
^, **, ↑, $	2	Right to Left
*, /, %	3	Left to Right
+, -	4	Left to Right

Note:
Evaluation of an arithmetic expression can be done in different ways, such as the manual method, the expression tree method, or the stack method. The most commonly used and crucial method in data structures is the stack method.

Evaluation of Infix Expression

Evaluation of an infix expression means the process of computing the final numerical result of a mathematical expression from left to right written in infix form.

<u>Algorithm Steps for Evaluation of Infix Expression using Stack Method:</u>

1. Initialize the two empty stacks, i.e., one for operators and another for operands.
2. Scan one element from the infix expression at a time from left to right.
3. If the scanned element is an operand, push it onto the operand stack.
4. If the scanned element is an operator, do the following:
   a. If the operator stack is empty, push the operator onto the operator stack.
   b. If the operator at the top of the stack has higher or equal precedence than the scanned operator:
   i. Pop that operator from the operator stack.
   ii. Perform the operation by popping the topmost element, calling it as 'S1' (right operand), and the next topmost element, calling it as 'S2' (left operand), from the operand stack. Like this: S2 op S1 / L op R, and push this back to the operand stack. Then push the scanned operator onto the operator stack.
5. If the scanned element is an opening parenthesis '(', push it onto the operator stack.
6. If the scanned element is a closing parenthesis ')', pop pushed operators from the operator stack and perform the operation by popping the topmost element, calling it as 'S1', and the next topmost element, calling it as 'S2', from the operand stack. Like this: S2 op S1. Then push this onto the operand stack. Continue until the opening parenthesis '(' occurs, and pop '(' also, but ')' is discarded.
7. Repeat the process for all elements in the expression.
8. After scanning all elements, apply the remaining operators from the operator stack until it becomes empty.
9. At the final step, pop the topmost element from the operand stack, which is the required evaluated infix result.

Evaluation of Postfix Expression

Evaluation of a postfix expression means the process of computing the final numerical result of a mathematical expression from left to right written in postfix form.

<u>Algorithm Steps for Evaluation of Postfix Expression using Stack Method:</u>

1. Initialize an empty stack.
2. Scan one element from the postfix expression at a time from left to right.
3. If the scanned element is an operand, push it onto stack.
4. If the scanned element is an operator, do the following:
   a. Pop the topmost element, calling it as 'S1' (right operand) from the stack.
   b. Pop the next topmost element, calling it as 'S2' (left operand) from the stack.
   c. Perform/ Evaluate the operation:
      S2 op S1 / L op R
   And push this back onto stack.
5. Repeat the steps until the entire expression is scanned.
6. If no element in the expression is left to scan, stop the process. After that, pop the topmost element from the stack, which is the required evaluated postfix result.

Evaluation of Prefix Expression

Evaluation of a prefix expression means the process of computing the final numerical result of a mathematical expression from right to left written in prefix form.

<u>Algorithm Steps for Evaluation of Prefix Expression using Stack Method:</u>

1. Initialize an empty stack.
2. Scan one element from the prefix expression at a time from right to left.
3. If the scanned element is an operand, push it onto stack.
4. If the scanned element is an operator, do the following:
   a. Pop the topmost element, calling it as 'S1' (right operand) from the stack.
   b. Pop the next topmost element, calling it as 'S2' (left operand) from the stack.
   c. Perform/ Evaluate the operation:
      S1 op S2 / R op L
      And push this back onto stack.
7. Repeat the steps until the entire expression is scanned.
8. If no element in the expression is left to scan, stop the process. After that, pop the topmost element from the stack, which is the required evaluated prefix result.

Expression Conversion:

Expression Conversion is the process of changing an arithmetic or logical formula from one notation to another, typically between infix, prefix, and postfix, or transforming data types within a formula to ensure proper computation. Expression conversion is performed based on the order of operator precedence and associativity.

Conversion from Infix Expression to Prefix Expression:

Infix Exp. to Prefix Exp. Conversion refers to the process of converting an infix expression (where operators are placed between operands) into a prefix expression (where operators are placed before the operands).

<u>Algorithm Steps for Infix Exp. to Prefix Exp. Conversion using Stack Method:</u>

1. Initialize an empty stack.
2. Scan one element from the infix expression at a time from left to right.
3. If the scanned element is an operand, append it to the output expression.
4. If the scanned element is an operator, do the following:
   a. If the stack is empty, push the operator onto the stack.
   b. If the operator at the top of the stack has higher or equal precedence than the scanned operator, pop that operator from the stack and append it to the output expression. Then push the scanned operator onto the stack.
5. If the scanned element is an opening parenthesis '(', push it onto the stack.
6. If the scanned element is a closing parenthesis ')', pop all the pushed operators from the stack (from right to left) until the opening parenthesis '(' occurs, and append them to the output expression. Also pop '(' but ')' is discarded.
7. Repeat the above steps until the entire expression is scanned.
8. If no element in the infix expression is left to scan, stop the process. After that, pop the remaining operators from the stack (from right to left) and append them to the output expression.

Conversion from Infix Expression to Prefix Expression:

Infix Expression to Prefix Expression Conversion refers to the process of converting an infix expression (where operators are placed between operands) into a prefix expression (where operators are placed before the operands).

<u>Algorithm Steps for Infix Exp. to Prefix Exp. Conversion using Stack Method:</u>

1. Convert each element of the given infix expression one by one from right to left to obtain the reversed infix expression.
2. Swap the parentheses/brackets such that '(' becomes ')' and ')' becomes '('.
3. Convert this modified expression into postfix form.
4. Then convert each element of the postfix result one by one from right to left to obtain the reversed postfix expression, which is the corresponding prefix form.

Conversion from Postfix Expression to Prefix Expression:

Postfix Expression to Prefix Expression Conversion refers to the process of converting an postfix expression (where operators are placed after the operands) into an infix expression (where operators are placed between operands).

<u>Algorithm Steps for Postfix Exp. to Infix Exp. Conversion using Stack Method:</u>

1. Initialize an empty stack.
2. Scan one element from the postfix expression at a time from left to right.
3. If the scanned element is an operand, push it onto the stack.
4. If the scanned element is an operator, do the following:
   a. Pop the topmost element, calling it as 'S1' (right operand) from the stack.
   b. Pop the next topmost element, calling it as 'S2' (left operand) from the stack.
   c. Concatenate them into a single unit with the operator in the middle and wrap them in parentheses:
      (S2 op S1) / (L op R)
      Then push this new string element back onto the stack
5. Repeat the above steps until the entire expression is scanned.
6. If no element in the postfix expression is left to scan, stop the process. Pop the topmost element from the stack, which is the required infix expression.

Conversion from Postfix Expression to Prefix Expression:

Postfix Expression to Prefix Expression Conversion refers to the process of converting a postfix expression (where operators are placed after operands) into a prefix expression (where operators are placed before operands).

<u>Algorithm Steps for Postfix Exp. to Prefix Exp. Conversion using Stack Method:</u>

1. Initialize an empty stack.
2. Scan one element from the postfix expression at a time from left to right.
3. If the scanned element is an operand, push it onto the stack.
4. If the scanned element is an operator, do the following:
   a. Pop the topmost element, calling it as S1 (right operand) from the stack.
   b. Pop the next topmost element, calling it S2 (left operand) from the stack.
   c. Concatenate them into a single unit with the operator in prefix order:
      op S2 S1 / op L R
      Then push this new string element back onto the stack.
5. Repeat the above steps until the entire expression is scanned.
6. If no element in the postfix expression is left to scan, stop the process. Pop the topmost element from the stack, which is the required prefix expression.

Conversion from Prefix Expression to Infix Expression:

Prefix Expression to Infix Expression Conversion refers to the process of converting a prefix expression (where operators are placed before the operands) into an infix expression (where operators are placed between operands).

Algorithm Steps for Prefix Exp. to Infix Exp. Conversion using Stack Method:

1. Initialize an empty stack.
2. Scan one element from the prefix expression at a time from right to left.
3. If the scanned element is an operand, push it onto the stack.
4. If the scanned element is an operator, do the following:
   a. Pop the topmost element, calling it as S1 (right operand) from the stack.
   b. Pop the next topmost element, calling it as S2 (left operand) from the stack.
   c. Concatenate them into a single unit with the operator in the middle and wrap them in parentheses:
      (S1 op S2) / (R op L)
      Then push this new string element back onto the stack.
5. Repeat the above steps until the entire expression is scanned.
6. If no element in the prefix expression is left to scan, stop the process. Pop the topmost element from the stack, which is the required infix expression.

Conversion from Prefix Expression to Postfix Expression:

Prefix Expression to Postfix Expression Conversion refers to the process of converting a prefix expression (where operators are placed before operands) into a postfix expression (where operators are placed after operands).

Algorithm Steps for Prefix Exp. to Postfix Exp. Conversion using Stack Method:

1. Initialize an empty stack.
2. Scan one element from the prefix expression at a time from right to left.
3. If the scanned element is an operand, push it onto the stack.
4. If the scanned element is an operator, do the following:
   a. Pop the topmost element, calling it as S1 (left operand) from the stack.
   b. Pop the next topmost element, calling it as S2 (right operand) from the stack.
   c. Concatenate them into a single unit in postfix order:
      S1 S2 op / R L op
      Then push this new string element back onto the stack.
5. Repeat the above steps until the entire expression is scanned.
6. If no element in the prefix expression is left to scan, stop the process. Pop the topmost element from the stack, which is the required postfix expression.

Note:
We reverse only in conversion problems, like:
1. Infix->Prefix
2. Prefix->Postfix (sometimes for easier method)
But for evaluation never required